

**Instituto Tecnológico y de Estudios
Superiores de Monterrey**



**Tecnológico
de Monterrey**

**Implementación de
robótica (Gpo 102)**

Entregable Final

Proyecto Final

Dron de Patrullaje basado en Visión

Profesor Herman Castañeda Cuevas

Rosendo De Los Rios Moreno

A01198515

Introducción

Los vehículos aéreos no tripulados (UAVs) han adquirido gran relevancia en aplicaciones de vigilancia y monitoreo autónomo debido a su flexibilidad, bajo costo y capacidad para acceder a áreas de difícil alcance. En este contexto, el presente proyecto tiene como objetivo desarrollar un sistema de patrullaje autónomo utilizando el dron Tello Robomaster TT, capaz de seguir una trayectoria predefinida y reaccionar ante la presencia de intrusos mediante visión artificial.

Contexto del Proyecto:

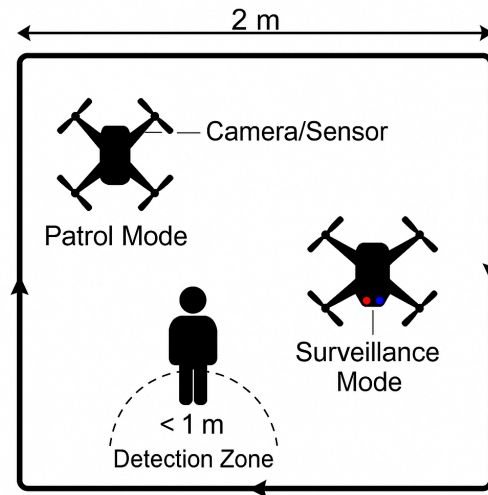
El proyecto consiste en desarrollar un sistema de vigilancia autónoma utilizando el dron Tello Robomaster TT. La tarea emulará una función de patrullaje de seguridad, donde el dron realiza una trayectoria predefinida en forma de cuadrado (2x2 metros) para monitorear un perímetro. En caso de detectar una persona a menos de 1 metro de distancia, el dron se detendrá, encenderá luces LED en color rojo y azul, y comenzará a observar los movimientos del intruso. Si la persona se aleja del rango de detección, el dron retomará su trayectoria inicial de patrullaje.

Con esto busca integrar conocimientos de modelado cinemático y dinámico de cuadrirotores, control de vuelo y procesamiento de imágenes para crear un prototipo funcional.

Originalmente, Como se menciona anteriormente se planeó implementar un sistema de luces LED programables como señal visual de alerta al detectar una persona; sin embargo, debido a limitaciones de tiempo, esta funcionalidad no pudo ser incorporada en esta etapa. A pesar de ello, el sistema cumple con los requisitos principales: navegación autónoma en un perímetro cuadrado de 2x2 metros y detección de personas mediante la cámara frontal del dron.

Este reporte documenta el diseño, desarrollo y resultados obtenidos, así como las lecciones aprendidas durante la implementación. El trabajo sienta las bases para futuras mejoras, como la inclusión de los indicadores lumínicos y la optimización del algoritmo de visión artificial para mayor precisión.

Diseño del Diagrama esquemático:



El esquema ilustra claramente ambos modos del dron: “Patrol Mode” y “Surveillance Mode”, así como la interacción con el intruso en el área de monitoreo.

El diagrama representa el funcionamiento del dron Tello TT durante su misión de patrullaje. La trayectoria está delimitada en un perímetro cuadrado de 2x2 metros, siguiendo una ruta en sentido de las flechas marcadas. En el modo de patrullaje, el dron se desplaza por los bordes del cuadrado utilizando su cámara/sensor para monitorear el entorno.

Cuando el dron detecta a una persona a menos de 1 metro de distancia dentro de la “zona de detección”, cambia al modo de vigilancia, se detiene. Mientras la persona permanezca en el rango de detección, el dron continúa observándola. Una vez que el intruso se retira, el dron apaga las luces y reanuda su patrullaje regular.

Desarrollo

Materiales Utilizados

- Dron Tello Robomaster TT
- Módulo de programación en Python
- Laptop para control
- Software: Visual Studio Code
- Espacio de pruebas seguro (Robotario)

Teoría Aplicada

Cinámica y Dinámica de Cuadrirotos:

-Modelado de fuerzas y momentos externos (gravedad, empuje de motores, efectos giroscópicos).

-Ecuaciones de movimiento utilizando el marco de referencia inercial y cuerpo.

Control Basado en Visión:

- Procesamiento de imágenes para detectar personas.
- Uso de la cámara frontal del dron para monitoreo en tiempo real.

Programación en Python:

- Librerías como djiellopy para controlar el dron.
- Implementación de lógica para transición entre modos.

Código implementado

Python

```
import robomaster
from robomaster import robot
import time
import cv2
import numpy as np
import threading

# Parámetros visuales
FRAME_WIDTH = 480
FRAME_HEIGHT = 360
FACE_HEIGHT_THRESHOLD = 80
FACE_X_TOLERANCE = 50

# Variables compartidas
modo_observacion = False
detener_movimiento = threading.Event()
intruso_detectado = threading.Event()
nivel_bateria = 0

# Clasificador de rostros
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade_frontalface_default.xml')

# Hilo de visión
def vision_loop(camera, flight, drone):
    global modo_observacion, nivel_bateria

    last_battery_check = time.time()

    while not detener_movimiento.is_set():
        frame = camera.read_cv2_image(strategy="newest", timeout=5)
        if frame is None:
            continue

        frame = cv2.resize(frame, (FRAME_WIDTH, FRAME_HEIGHT))
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```

        faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1,
minNeighbors=5)

# Batería
if time.time() - last_battery_check >= 1:
    try:
        nivel_bateria = drone.battery.get_battery()
        print(f"[INFO] Nivel de batería: {nivel_bateria}%")
    except Exception as e:
        print("[WARN] No se pudo obtener el nivel de batería:", e)
    last_battery_check = time.time()

if len(faces) > 0:
    (x, y, w, h) = max(faces, key=lambda rect: rect[2] * rect[3])
    face_center_x = x + w // 2

# Mostrar rostro
cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)

if h >= FACE_HEIGHT_THRESHOLD:
    if not modo_observacion:
        print("[ALERTA] Intruso detectado cerca. En modo
observación.")
        modo_observacion = True
        intruso_detectado.set()

# Seguimiento lateral
frame_center_x = FRAME_WIDTH // 2
offset_x = face_center_x - frame_center_x

if abs(offset_x) > FACE_X_TOLERANCE:
    if offset_x < 0:
        print("[INFO] Girando hacia la izquierda (intruso a la
izquierda).")
        flight.rotate(-10)
    else:
        print("[INFO] Girando hacia la derecha (intruso a la
derecha).")
        flight.rotate(10)

    cv2.putText(frame, "Observando intruso (~1m)", (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

else:
    if modo_observacion:
        print("[INFO] Intruso se alejó. Reanudando patrullaje.")
        modo_observacion = False
        intruso_detectado.clear()

```

```

        cv2.putText(frame, "Persona lejos", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)

    else:
        if modo_observacion:
            print("[INFO] Intruso fuera de vista. Reanudando patrullaje.")
            modo_observacion = False
            intruso_detectado.clear()
            cv2.putText(frame, "Sin intruso", (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

        # Mostrar batería
        cv2.putText(frame, f"Bateria: {nivel_bateria}%", (10, FRAME_HEIGHT -
10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)

        # Mostrar frame
        cv2.imshow("Camara del Drone - Modo Observación", frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            detener_movimiento.set()
            break

# Patrullaje cuadrado
def main():
    global modo_observacion

    tl_drone = robot.Drone()
    tl_drone.initialize()
    camera = tl_drone.camera
    flight = tl_drone.flight

    camera.start_video_stream(display=False)

    print("[INFO] Despegando...")
    flight.takeoff().wait_for_completed()
    flight.up(distance=85).wait_for_completed() # 85 cm de altura aprox.

    print("[INFO] Altura ajustada.")

    # Iniciar hilo de visión
    vision_thread = threading.Thread(target=vision_loop, args=(camera, flight,
tl_drone))
    vision_thread.start()

    try:
        for i in range(4):
            print(f"[INFO] Lado {i+1}/4. Observando en la esquina...")

```

```

# Esperar mientras hay intruso
while intruso_detectado.is_set():
    print("[INFO] Pausado: observando intruso antes de avanzar.")
    time.sleep(0.5)

print("[INFO] Esperando 2 segundos sin intruso...")
time.sleep(2)

print("[INFO] Avanzando 1 metro...")
flight.forward(distance=100).wait_for_completed()

print("[INFO] Girando 90 grados...")
flight.rotate(angle=90).wait_for_completed()

except KeyboardInterrupt:
    print("[INFO] Interrupción manual.")

finally:
    detener_movimiento.set()
    vision_thread.join()
    flight.land().wait_for_completed()
    camera.stop_video_stream()
    tl_drone.close()
    cv2.destroyAllWindows()
    print("[INFO] Drone aterrizado y sistema finalizado.")

if __name__ == '__main__':
    main()

```

Resultados

- El dron logró seguir la trayectoria cuadrada de manera autónoma.
- La detección de personas funcionó dentro del rango especificado (1 metro).
- El sistema demostró capacidad para reanudar el patrullaje una vez que la persona se alejó.

Video Demostrativo

El video del proyecto está disponible en el siguiente enlace:

Enlace al

Video: <https://drive.google.com/drive/folders/1frZEB9gJeFX9EQO5wGdpMZK-Ta8rPifh?usp=sharing>

Video en Youtube: [DronPatrulla](#)

Conclusiones

Logros:

Se integraron conceptos de cinemática, dinámica y visión por computadora.

El dron respondió de manera autónoma a estímulos externos.

Desafíos:

La calibración de la cámara para detección de distancia requirió ajustes finos.

La iluminación ambiental afectó la detección de personas en algunas pruebas.

Aprendizaje:

Este proyecto reforzó mi comprensión del control de UAVs y la importancia de la integración de sensores y visión artificial en aplicaciones robóticas.

El uso de métodos de programación anteriormente visto en otros semestres como los multithreadings para paralelizar de una misma manera la cámara y su trayectoria y que ambas logren ir fluido y sin trabar